

BACKEND PRODUCT REQUIREMENTS

Femi9 Platform

Turning the Femi9 storefront from a static front-end into a production, end-to-end commerce & operations system — with a real database, APIs, authentication, payments, and a custom admin.

DOCUMENT	Backend PRD (workflow & architecture)
VERSION	Draft v1.0 — for review
DATE	18 July 2026
PRODUCT	femi9-lavender (live storefront)
TARGET STACK	Next.js (full-stack) · PostgreSQL · Razorpay · custom admin
STATUS	Awaiting founder review → task breakdown

This PRD describes *what* to build and the workflows between parts. It is intentionally implementation-light so it can be reviewed and split into tasks before any code is written.

Contents

- | | |
|-----------------------------------|---|
| 1 Purpose & scope | 13 Affiliate program workflow |
| 2 Current state & the gap | 14 Partner (reseller) workflow |
| 3 Target architecture | 15 Community wall & moderation |
| 4 Technology stack | 16 Cycle tracking & predictions |
| 5 Environments & infrastructure | 17 Blog / content CMS |
| 6 Data model (schema) | 18 Admin panel specification |
| 7 Authentication & roles | 19 Analytics & reporting |
| 8 API surface | 20 Notifications (email / SMS / WhatsApp) |
| 9 Commerce workflow: cart → order | 21 Non-functional requirements |
| 10 Payments (Razorpay) workflow | 22 Data migration & known fixes |
| 11 Subscriptions workflow | 23 Delivery roadmap (phases) |
| 12 Rewards / loyalty ledger | 24 Open decisions |

1 Purpose & scope

Give Femi9 a real backend so the storefront stops being a demo and becomes a system real customers, creators, resellers, and staff can use.

The Femi9 storefront (`femi9-lavender`) is a polished React SPA, but every page reads from static files and browser `localStorage` . There is no database, no login, no real orders, and no way for staff to change anything without a code deploy. This document specifies the backend and the workflows that connect the storefront, the customers, and the operators.

In scope

- Persistent database and data model for every entity the site already implies.
- Real authentication with roles (customer, admin, affiliate, partner) and protected routes.
- Server-side APIs replacing every static `src/data/*` import and every `localStorage` write.
- Commerce: cart with variants, orders, inventory, online payments via Razorpay, subscriptions.
- A custom admin panel to manage catalog, orders, content, users, and programs.
- Domain programs: rewards ledger, affiliate attribution & payouts, partner lead intake, community wall moderation, cycle tracking.
- Deployment, environments, secrets, and operational concerns.

Out of scope (for this PRD)

- Visual redesign of the storefront — the existing UI stays; only its data source changes.
- The mobile app (`femi9-app`), the legacy static site, and the standalone scroll component.
- A native ML training pipeline — the "period-prediction model" is treated as a computed/heuristic service with a defined interface, not a data-science project.

2 Current state & the gap

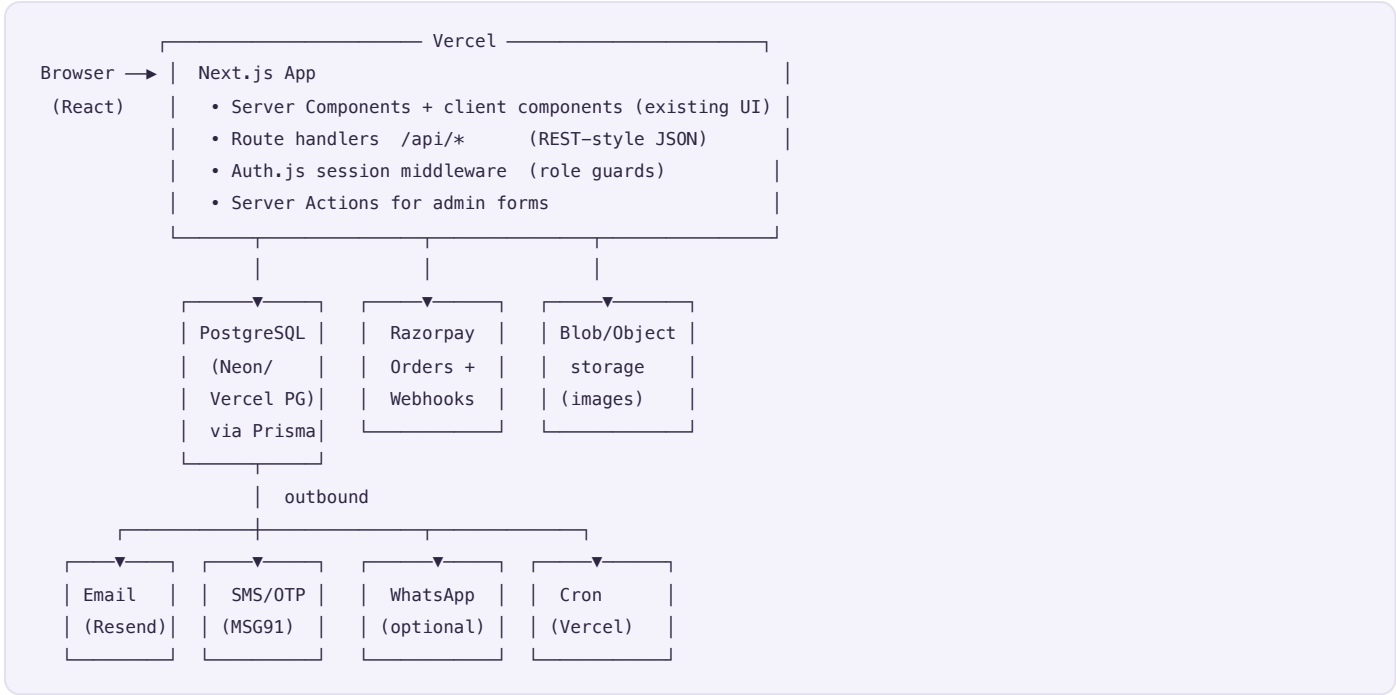
Audit of the live codebase found a complete front-end shell with no backing services. The table below is the gap this PRD closes.

Capability	Today	Target
Content (products, blog, etc.)	Hardcoded in <code>src/data/*.ts</code>	Database rows, editable in admin, served by API
Authentication	None. One hardcoded user "Aishwarya Menon"	Real accounts, sessions, phone/email login
Protected pages	<code>/account</code> , <code>/dashboard</code> , <code>/admin</code> open to anyone	Role-gated, server-enforced
Cart	In-memory React state, lost on refresh; no variants	Persisted cart with pack/size variants
Checkout / orders	WhatsApp deep-link; no order is stored	Real orders in DB + online payment
Payments	None	Razorpay (UPI / cards / netbanking) + webhooks
Admin	Read-only charts only	Full CRUD + operations console
Rewards / affiliate / partner / wall	<code>localStorage</code> , per-browser, not shared	Server records, real attribution & moderation
Analytics	Hardcoded numbers	Aggregated from real orders & events

Two data bugs to fix during migration. (1) Order line-items currently reference a product by its display *name* string instead of an id — the schema below standardises on id foreign keys. (2) The cart cannot represent pack-size or panty-size variants that the product model supports — the new cart/order line captures a `variant` .

3 Target architecture

Per the chosen direction, the storefront migrates from a Vite SPA to a **Next.js full-stack app** deployed on Vercel. Next.js hosts both the rendered site and the API (route handlers), so front-end and backend ship as one project with one deploy.



Key principles

- **One codebase.** The admin, the storefront, and the API live in the same Next.js project, sharing types and the Prisma client.
- **Server owns the truth.** Prices, inventory, points, and order totals are computed and validated server-side — never trusted from the client.
- **Everything through a service layer.** Route handlers are thin; business rules live in typed service modules that both API routes and server actions call.
- **Migration seam.** Each static `src/data/*` file maps to a table + an endpoint; pages are cut over one domain at a time.

4 Technology stack

Framework	Next.js (App Router), React 18, TypeScript — migrated from the current Vite SPA
Database	PostgreSQL — Neon or Vercel Postgres (serverless-friendly, integrates with Vercel)
ORM / migrations	Prisma (schema-first, typed client, versioned migrations)
Auth	Auth.js (NextAuth v5): email magic-link + phone OTP; session in secure cookie; role claim
Payments	Razorpay — Orders API, Checkout, webhook signature verification, refunds
Validation	Zod schemas shared by client forms and server handlers
File storage	Vercel Blob or Cloudinary/S3 — product & blog images, review photos
Transactional email	Resend (order confirmations, magic links, coupon codes)
SMS / OTP	MSG91 or Twilio (India OTP, order updates)
Background / scheduled	Vercel Cron (subscription renewals, digest emails, low-stock alerts)
Observability	Vercel logs + Sentry (errors), structured request logging
Rate limiting	Upstash Redis (OTP + write-endpoint throttling)

Why these: every choice is serverless-compatible and Vercel-native, so the platform runs with near-zero ops and a free/low tier during launch, then scales without re-architecture. All are swappable behind the service layer if a vendor changes.

5 Environments & infrastructure

Environment	Purpose	Data	Payments
Local	Development	Local/branch Postgres, seeded from migration	Razorpay test keys
Preview	Per-PR review on Vercel	Ephemeral branch DB (Neon branch)	Razorpay test keys
Production	Live site	Production Postgres, backups on	Razorpay live keys

Secrets / environment variables (managed in Vercel, never committed)

```
DATABASE_URL / DIRECT_URL      # Postgres (pooled + direct for migrations)
AUTH_SECRET                    # session signing
RAZORPAY_KEY_ID / RAZORPAY_KEY_SECRET
RAZORPAY_WEBHOOK_SECRET
RESEND_API_KEY                 # email
MSG91_AUTH_KEY (or TWILIO_*)  # SMS / OTP
BLOB_READ_WRITE_TOKEN          # object storage
UPSTASH_REDIS_URL / _TOKEN     # rate limiting
SENTRY_DSN
NEXT_PUBLIC_RAZORPAY_KEY_ID    # public checkout key
NEXT_PUBLIC_SITE_URL
```

Founder action needed. These require *your* accounts and cannot be created on your behalf: Razorpay (KYC + bank), the database provider, the email/SMS providers, and the domain. The build can proceed against test keys; going live is gated on these.

6 Data model (schema)

The relational model below covers every entity found in the current code. Types are indicative (Prisma/Postgres). PK = primary key, FK = foreign key.

Catalog

Table	Key fields	Notes
product	id (PK), slug, name, type(pad panty), basePrice, meta, flow, description, longDescription, tag, rating, reviewCount, status(active draft archived), createdAt	Core catalog item
product_variant	id (PK), productId (FK), kind(pack size), label, packCount, size, price, sku, stock, active	Fixes the "cart can't hold variants" gap; inventory lives here
product_image	id (PK), productId (FK), url, alt, position	Gallery, ordered
product_feature	id (PK), productId (FK), title, body, position	Detail "features" list
product_spec	id (PK), productId (FK), key, value, position	Spec rows
cadence	id (PK), label, sub, days, active	Subscription frequency options

Customers & identity

Table	Key fields	Notes
user	id (PK), name, email(unique), phone(unique), emailVerified, role(customer admin affiliate partner staff), tier, createdAt	Replaces the hardcoded user; role drives access
account / session	Auth.js tables	Sessions, OTP challenges, provider links
address	id (PK), userId (FK), label, name, line, city, state, pincode, phone, isPrimary	Shipping/billing

Commerce

Table	Key fields	Notes
cart	id (PK), userId (FK, nullable for guest), createdAt, updatedAt	Server-persisted; guest carts by cookie token
cart_item	id (PK), cartId (FK), variantId (FK), qty	Variant-aware line
order	id (PK), orderNo, userId (FK), status(pending paid processing shipped delivered cancelled refunded), subtotal, discount, shipping, total, addressId (FK), couponId, affiliateCodeId, placedAt	Immutable financial record
order_item	id (PK), orderId (FK), variantId (FK), productName, variantLabel, unitPrice, qty, lineTotal	Snapshots name/price at purchase (by id, not name)
payment	id (PK), orderId (FK), provider, razorpayOrderId, razorpayPaymentId, amount, status, method, signatureVerified, raw(JSON), createdAt	One per attempt; webhook-updated
coupon	id (PK), code, type(flat pct), value, minOrder, maxUses, usedCount, expiresAt, active	Rewards redemptions issue these
subscription	id (PK), userId (FK), variantId (FK), qty, cadenceId (FK), status(active paused cancelled), nextDeliveryAt, savedTotal	Recurring orders

Loyalty, programs & community

Table	Key fields	Notes
points_ledger	id (PK), userId (FK), delta(+/-), reason, orderId?, balanceAfter, createdAt	Append-only; balance = sum. Replaces localStorage rewards
reward_option	id (PK), title, costPoints, couponType, couponValue, active	Redeemable catalogue
affiliate	id (PK), userId (FK), handle, platform, followerBand, promoCode(unique), status(pending approved suspended), payoutUpi, createdAt	Creator account
affiliate_event	id (PK), affiliateId (FK), type(click signup order), orderId?, amount, commission, createdAt	Attribution & earnings
affiliate_payout	id (PK), affiliateId (FK), amount, status, periodStart/End, paidAt	Payout runs
partner_application	id (PK), name, phone, city, situation, reason, status(new contacted onboarded rejected), notes, createdAt	Reseller lead intake
wall_post	id (PK), userId?(nullable), alias, isAnonymous, productId?, rating, body, tags(string[]), status(pending approved hidden), likeCount, createdAt	Community stories; moderated
wall_like / wall_reply	id (PK), postId (FK), userId, ...	Persisted engagement
review	id (PK), productId (FK), userId?, orderId?, name, place, rating, body, status, createdAt	Product reviews (currently sample data)

Cycle tracking & content

Table	Key fields	Notes
period_log	id (PK), userId (FK), startDate, lengthDays	Raw source; predictions are computed from this
symptom_log	id (PK), userId (FK), date, symptom, level(0-3)	Per-day symptom entries
blog_post	id (PK), slug, title, category, excerpt, author, body(markdown/blocks), tone, image, featured, status, publishedAt	CMS-managed
blog_category	id (PK), name, color, tint	Lookup
event_log	id (PK), type, userId?, meta(JSON), createdAt	Feeds analytics (orders, views, signups)

Analytics is derived, not stored. The admin KPIs, revenue trend, product mix, orders-per-month, regional demand, and forecasts are computed by queries/rollups over `order`, `order_item`, `event_log`, and `period_log` — not hand-entered like today.

7 Authentication & roles

Methods

- **Phone OTP** (primary for India) — enter phone → SMS code → verify → session.
- **Email magic-link** — passwordless, for desktop/creators/admin.
- No passwords stored. OTP challenges are rate-limited and single-use with short expiry.

Roles & access

Role	Can access
customer	Storefront, own <code>/account</code> , <code>/dashboard</code> , orders, subscriptions, cycle data, rewards, wall posting
affiliate	Customer scope + affiliate dashboard (own code, events, earnings, payout details)
partner	Customer scope + partner portal (application status, resources)
staff	Admin panel with limited modules (e.g. orders, moderation) — no settings/payouts
admin	Full admin panel including catalog, users, payouts, coupons, settings

Enforcement

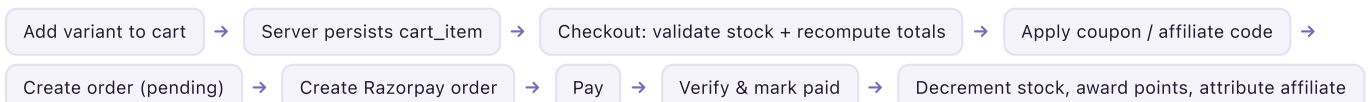
- Middleware guards route groups: `/admin` requires `staff|admin`; `/account` requires any authenticated user; ownership checks on every record (a user can only read their own orders/cycle data).
- Server-side checks on every mutating endpoint — never rely on hidden UI. This closes today's "open `/admin`" hole.

8 API surface

REST-style route handlers under `/api`; admin mutations may also use server actions. Representative endpoints (not exhaustive):

Group	Endpoints	Auth
Catalog	<code>GET /api/products</code> , <code>GET /api/products/:slug</code> , <code>GET /api/products/:id/reviews</code>	Public
Cart	<code>GET/POST /api/cart</code> , <code>PATCH/DELETE /api/cart/items:id</code>	Session/guest
Checkout	<code>POST /api/checkout</code> (creates order + Razorpay order), <code>POST /api/checkout/verify</code>	Session
Payments	<code>POST /api/webhooks/razorpay</code>	Signature
Orders	<code>GET /api/orders</code> , <code>GET /api/orders:no</code>	Owner
Subscriptions	<code>GET/POST /api/subscriptions</code> , <code>PATCH /api/subscriptions/:id</code> (pause/skip/cancel)	Owner
Rewards	<code>GET /api/rewards</code> , <code>POST /api/rewards/redeem</code>	Owner
Cycle	<code>GET /api/cycle</code> , <code>POST /api/cycle/periods</code> , <code>POST /api/cycle/symptoms</code>	Owner
Affiliate	<code>POST /api/affiliate/apply</code> , <code>GET /api/affiliate/me</code> , <code>GET /r/:code</code> (redirect + click)	Mixed
Partner	<code>POST /api/partner/apply</code>	Public
Wall	<code>GET /api/wall</code> , <code>POST /api/wall</code> , <code>POST /api/wall/:id/like</code>	Public read / session write
Blog	<code>GET /api/blog</code> , <code>GET /api/blog/:slug</code>	Public
Admin	<code>/api/admin/*</code> CRUD for every entity + <code>/api/admin/analytics/*</code>	staff/admin

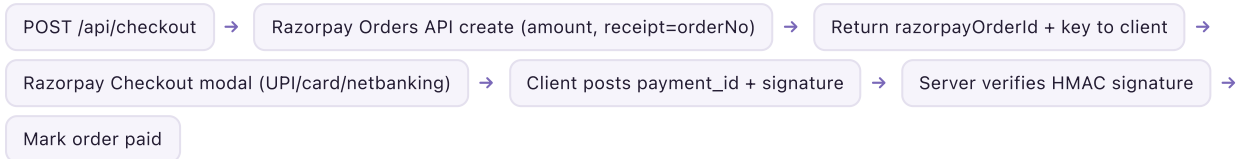
9 Commerce workflow: cart → order



- **Server recomputes every price** from `product_variant.price` at checkout; the client-sent amount is ignored.
- Stock is checked at checkout and decremented only on confirmed payment (inside a DB transaction to avoid overselling).

- Free-shipping threshold, subscribe-and-save %, and coupon logic run server-side as pricing rules.
- The existing WhatsApp handoff is retained as a fallback "order on WhatsApp" button but is no longer the primary path.

10 Payments (Razorpay) workflow



- **Two confirmation paths, both required:** synchronous signature verification on return *and* the asynchronous `razorpay` webhook (source of truth). The order is fulfilled only when a verified `payment.captured` is recorded — idempotent on `razorpayPaymentId`.
- Webhook endpoint verifies `X-Razorpay-Signature` against `RAZORPAY_WEBHOOK_SECRET`; unverified calls are rejected.
- Failure/timeout leaves the order `pending`; a reconciliation cron polls Razorpay for stuck orders.
- Refunds issued from admin call Razorpay Refunds API and write a `refunded` status + points reversal.

Compliance: the app never stores card/UPI credentials — Razorpay Checkout collects them. We store only Razorpay ids, amounts, and status. This keeps Femi9 out of PCI scope.

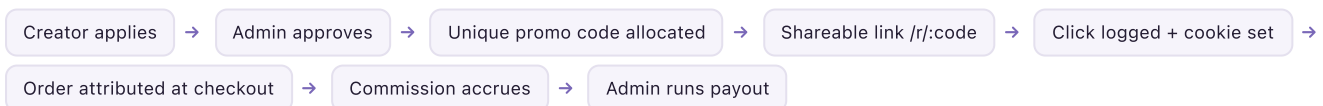
11 Subscriptions workflow

- A subscription binds a user + variant + qty + cadence (every cycle / 4w / 6w) with a `nextDeliveryAt`.
- A **Vercel Cron** runs daily, finds subscriptions due, and generates a new order (reusing the checkout pipeline). Payment for renewals uses Razorpay recurring/tokenised payments or a prompt-to-pay link (decision in §24).
- Users can **pause, skip next, change qty/cadence, or cancel** from `/account`; each writes to the subscription and re-schedules.
- "Saved so far" is computed from the subscribe-and-save discount across generated orders — no longer a static number.

12 Rewards / loyalty ledger

- Points are an **append-only ledger** (`points_ledger`); the balance is the running sum — auditable, no lost updates. Replaces the per-browser `localStorage` balance.
- **Earn** rules run server-side: e.g. points per order, first-order bonus, review/wall-post bonus, referral. Each writes a positive ledger row on the triggering event.
- **Redeem:** user spends points on a `reward_option` → a negative ledger row + a real `coupon` is issued and emailed. Redemption is transactional (no double-spend).
- Tier ("Bloom member") derived from lifetime points or spend thresholds.

13 Affiliate program workflow



- Promo codes are **server-allocated and unique** (today they are a client-side hash — collision-prone and unverifiable).
- Attribution: `/r/:code` logs a click and sets a first-party attribution cookie; if a matching code is applied or the cookie is present at checkout, the order links to the affiliate and commission is computed per tier.
- Affiliate dashboard shows *real* clicks / orders / earnings from `affiliate_event` — not zeros.
- Payouts: admin generates a payout run for a period; records UPI reference; marks paid. (Automated UPI transfer is a later option.)

14 Partner (reseller) workflow

- Application form writes a `partner_application` row (name, phone, city, situation, reason) — replacing the `localStorage` -only save that no one at Femi9 can see today.
- On submit: confirmation to applicant (SMS/WhatsApp) + notification to the ops team; lead appears in the admin CRM board.
- Admin moves leads through `new` → `contacted` → `onboarded` / `rejected` with notes. Onboarded partners can be upgraded to the `partner` role for a resource portal + wholesale pricing (future).

15 Community wall & moderation

- Posts persist to `wall_post` (shared across all visitors, not one browser). Optional anonymity via `alias` / `isAnonymous`.
- **Moderation queue:** new posts default to `pending`; admin approves/hides. Optional auto-approval with profanity/keyword filtering for trusted users.

- Likes and replies persist server-side (today likes are in-memory only). Rate-limited to deter spam.
- Reporting: users can flag; flagged posts surface in the admin queue.

16 Cycle tracking & predictions

- Raw truth is `period_log` + `symptom_log` per user. All predictions are **computed server-side** from the log — average cycle/period length, next start, ovulation/fertile window, PMS window, confidence.
- A `CyclePredictionService` exposes one interface (`predict(userId)`) so today's heuristic can later be replaced by the ML model without changing callers.
- Cycle data is **sensitive personal/health data** — strict ownership checks, encrypted at rest (DB-level), never shared with affiliates/analytics except as anonymised aggregates.
- The admin "period-prediction model" panel (accuracy, restock recommendations, demand uplift) is fed by aggregated, de-identified cycle data across consenting users.

17 Blog / content CMS

- Blog posts move from a 700-line static file to `blog_post` rows, edited in admin with a rich body (markdown or block JSON preserving the current `##` heading / `>` pull-quote conventions).
- Draft → publish workflow with `publishedAt` ; categories managed in `blog_category` .
- Images upload to object storage; slugs are unique and drive `/blog/:slug` .
- Same CMS pattern extends to editable homepage sections (hero banners, impact stats) so marketing copy no longer needs a code deploy.

18 Admin panel specification

A custom, branded admin at `/admin` (role-gated), replacing today's read-only charts. Modules:

Module	Capabilities
Dashboard	Live KPIs, revenue & forecast, product mix, orders/month, regional demand — all from real data
Catalog	CRUD products, variants, images, features, specs; set price, stock, tags, status
Inventory	Stock levels, low-stock alerts, adjustments
Orders	List/search/filter, view detail, update status, print invoice, issue refunds, add tracking
Subscriptions	View active subs, upcoming renewals, pause/cancel on behalf
Customers	Search users, view orders/points/tier, adjust points, roles
Coupons	Create/expire codes, usage stats
Content	Blog CMS, homepage sections, reviews moderation
Community	Wall moderation queue, approve/hide, handle flags
Affiliates	Approve creators, view events/earnings, run payouts
Partners	Lead CRM board (new → contacted → onboarded), notes
Settings	Shipping thresholds, subscribe-save %, tax, reward rules, staff roles

"Customizable backend." Business parameters (free-ship threshold, subscribe-save %, points rules, cadences, reward options) live in editable settings/records — not hardcoded constants like today's `FREE_SHIP` / `SUBSCRIBE_PCT` — so staff tune the business without a deploy.

19 Analytics & reporting

- KPIs and charts currently shown in `/admin` are recomputed from real tables via aggregation queries (optionally materialised/cached for speed).
- Event tracking (`event_log`) captures product views, add-to-cart, checkout, signups — enabling funnels and conversion metrics.
- Regional demand, demand forecast, and cycle-length distribution derive from orders + (consented) cycle data.
- Exports (CSV) for orders, payouts, and inventory.

20 Notifications

Channel	Triggers
Email (Resend)	Magic-link login, order confirmation, shipping update, coupon/reward code, subscription reminders
SMS (MSG91/Twilio)	OTP login, order placed/shipped, partner application received
WhatsApp (optional)	Order updates, partner follow-up (retains the brand's current WhatsApp-first habit)

All sends go through a single notification service with templates, so channels are swappable and auditable.

21 Non-functional requirements

Area	Requirement
Security	Server-side authz on every mutation; ownership checks; input validated with Zod; secrets in env only; signed webhooks; no card data stored
Privacy	Cycle/health data encrypted & access-restricted; consent for aggregate use; data-export & delete on request; India DPDP-aware
Performance	Preserve current LCP (~950ms) & CLS 0; DB indexes on hot paths; cache catalog/blog; keep route code-splitting
Reliability	Payment idempotency; DB transactions for stock/points; webhook retries; nightly backups
Abuse control	Rate-limit OTP, checkout, wall/review writes; bot protection on public forms
Observability	Sentry errors, structured logs, payment/webhook audit trail
SEO	SSR/metadata for product & blog pages (a Next.js benefit); keep the SPA-rewrite behaviour gone (no more 404 workaround)

22 Data migration & known fixes

- Seed the database from the existing `src/data/*` files (5 products, 12 blog posts, cadences, categories) via a one-time migration script — no content is lost.
- Fix #1:** order line-items keyed by product `id` (was display name).
- Fix #2:** cart/order lines carry a `variantId` so pack-size and panty-size are captured.
- Migrate any real `localStorage` demo data patterns to their server tables (rewards, subscriptions, wall posts, affiliate, partner) — going forward these are server-owned.
- Replace hardcoded constants (`FREE_SHIP` , `SUBSCRIBE_PCT` , `WA_NUMBER`) with editable settings records.

23 Delivery roadmap (phases)

Each phase is independently shippable. Ordered so revenue-critical flows land first.

Phase	Scope	Outcome
PHASE 0 Foundation	Next.js migration of the existing SPA, Postgres + Prisma, schema + seed, CI/deploy pipeline	Same site, now full-stack & data-driven
PHASE 1 Auth & accounts	Phone OTP + email login, roles, route guards, real <code>/account</code> & <code>/dashboard</code> from DB	Real users replace the hardcoded one
PHASE 2 Commerce core	Variant-aware cart, checkout, orders, inventory, Razorpay + webhooks	The store can take real, paid orders
PHASE 3 Admin panel	Custom admin: catalog, orders, inventory, customers, coupons, settings, live analytics	Staff run the business without deploys
PHASE 4 Programs	Rewards ledger, affiliate attribution & payouts, partner CRM, community wall + moderation	Growth programs become real
PHASE 5 Lifecycle	Subscriptions + renewals cron, cycle tracking + predictions service, blog CMS, notifications	Retention & content self-serve
LAUNCH Hardening	Security review, load test, backups, Sentry, live Razorpay keys, domain cutover	Production go-live

24 Open decisions (need your input)

#	Decision	Recommendation
1	Database host — Neon vs Vercel Postgres vs Supabase Postgres	Neon (branching for previews, generous tier)
2	Subscription payments — tokenised auto-charge vs pay-link each cycle	Start with pay-link reminders; add auto-charge later (Razorpay recurring needs extra KYC)
3	Guest checkout allowed, or login required?	Allow guest checkout; create account post-purchase
4	OTP/SMS provider — MSG91 vs Twilio	MSG91 (India pricing & deliverability)
5	Image storage — Vercel Blob vs Cloudinary	Cloudinary (built-in image optimisation)
6	Keep WhatsApp order option alongside Razorpay?	Yes, as secondary — it matches current buyer habit
7	Do we need GST invoicing / tax at launch?	Confirm — affects order & pricing fields
8	Staff/admin user list at launch	Provide emails/phones to seed admin roles

Next step. Review this PRD, resolve the eight decisions above, and confirm the phase order. On approval this converts directly into an epic-per-phase task breakdown, and implementation can begin at Phase 0.